

third

March 1, 2026

1 Projektovanje konvolucione neuralne mreže za klasifikaciju slika

1.1 AI vs Human Generated Images

Opis problema: Klasifikacija slika u dve klase — slike generisane veštačkom inteligencijom (AI) i autentične slike koje su nastale fotografisanjem (Human). Model treba da nauči da prepoznaje razlike između ove dve kategorije na osnovu vizuelnih karakteristika.

1.2 1. Imports

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import os

from keras.utils import image_dataset_from_directory
from keras.models import Sequential
from keras import layers
from keras.callbacks import EarlyStopping
from keras.losses import SparseCategoricalCrossentropy

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report

print(f'TensorFlow version: {tf.__version__}')
print(f'GPU available: {len(tf.config.list_physical_devices("GPU")) > 0}')
```

```
2026-02-28 02:21:16.663588: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
```

```
2026-02-28 02:21:16.664033: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
2026-02-28 02:21:16.718235: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
```

```
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA,
```

in other operations, rebuild TensorFlow with the appropriate compiler flags.
2026-02-28 02:21:18.243627: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.
2026-02-28 02:21:18.244577: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.

TensorFlow version: 2.20.0

GPU available: False

2026-02-28 02:21:18.929336: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

1.3 2. Preuzimanje skupa podataka sa Kaggle

```
[2]: import subprocess
import os

# Postavljanje Kaggle API tokena
os.environ['KAGGLE_API_TOKEN'] = 'KGAT_fd2c5151f762387148d28c5a8dd2d08f'

DATA_DIR = './data'

if not os.path.exists(os.path.join(DATA_DIR, 'train_data')):
    print('Preuzimanje datasea...')
    subprocess.run([
        'kaggle', 'datasets', 'download',
        '-d', 'alessandrasala79/ai-vs-human-generated-dataset',
        '-p', DATA_DIR, '--unzip'
    ], check=True)
    print('Dataset preuzet i raspakovan.')
else:
    print('Dataset vec postoji.')
```

Dataset vec postoji.

```
[3]: # Pregled strukture foldera
for root, dirs, files in os.walk(DATA_DIR):
    depth = root.replace(DATA_DIR, '').count(os.sep)
    indent = ' ' * 2 * depth
    print(f'{indent}{os.path.basename(root)}/')
    if depth < 2:
        for d in dirs:
            subdir_path = os.path.join(root, d)
            n_files = len([f for f in os.listdir(subdir_path) if os.path.
↪isfile(os.path.join(subdir_path, f))])
```

```
print(f'{indent} {d}/ ({n_files} fajlova)')
```

```
data/  
  train_data/ (79950 fajlova)  
  test_data_v2/ (5540 fajlova)  
train_data/  
test_data_v2/
```

1.4 3. Učitavanje i eksploracija podataka

```
[4]: import pandas as pd  
  
# Parametri  
IMG_SIZE = (128, 128)  
BATCH_SIZE = 64  
SEED = 42  
  
DATA_DIR = './data'  
  
# Učitavanje labela iz CSV  
df = pd.read_csv(os.path.join(DATA_DIR, 'train.csv'), index_col=0)  
df['full_path'] = df['file_name'].apply(lambda x: os.path.join(DATA_DIR, x))  
df['label'] = df['label'].astype(int)  
  
# Shuffle i podela: 70% trening, 10% validacija, 20% test  
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)  
n = len(df)  
train_df = df.iloc[:int(0.7 * n)]  
val_df = df.iloc[int(0.7 * n):int(0.8 * n)]  
test_df = df.iloc[int(0.8 * n):]  
  
# Funkcija za kreiranje tf.data.Dataset iz dataframe-a  
def make_dataset(dataframe, shuffle=False):  
    paths = dataframe['full_path'].values  
    labels = dataframe['label'].values  
  
    def load_image(path, label):  
        img = tf.io.read_file(path)  
        img = tf.image.decode_jpeg(img, channels=3)  
        img = tf.image.resize(img, IMG_SIZE)  
        return img, label  
  
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))  
    if shuffle:  
        ds = ds.shuffle(buffer_size=2000, seed=SEED)  
    ds = ds.map(load_image, num_parallel_calls=tf.data.AUTOTUNE)  
    ds = ds.batch(BATCH_SIZE)
```

```

ds = ds.prefetch(tf.data.AUTOTUNE)
return ds

Xtrain = make_dataset(train_df, shuffle=True)
Xval   = make_dataset(val_df)
Xtest  = make_dataset(test_df)

# 0 = AI-generisane, 1 = Human (stvarne)
classes = ['AI', 'Human']
num_classes = 2

print(f'Trening skup:      {len(train_df)} slika')
print(f'Validacioni skup: {len(val_df)} slika')
print(f'Test skup:        {len(test_df)} slika')
print(f'Klase: {classes}')
print(f'Broj klasa: {num_classes}')

```

```

Trening skup:      55965 slika
Validacioni skup: 7995 slika
Test skup:        15990 slika
Klase: ['AI', 'Human']
Broj klasa: 2

```

```

[5]: # Prikaz jednog primerka iz svake klase
fig, axes = plt.subplots(1, num_classes, figsize=(5 * num_classes, 5))
if num_classes == 1:
    axes = [axes]

shown_classes = set()
for images, labels in Xtrain:
    for i in range(len(labels)):
        label = labels[i].numpy()
        if label not in shown_classes:
            ax = axes[label]
            ax.imshow(images[i].numpy().astype('uint8'))
            ax.set_title(classes[label], fontsize=14)
            ax.axis('off')
            shown_classes.add(label)
        if len(shown_classes) == num_classes:
            break
    if len(shown_classes) == num_classes:
        break

plt.suptitle('Jedan primerak iz svake klase', fontsize=16)
plt.tight_layout()
plt.show()

```

Jedan primerak iz svake klase

AI



Human



```
[6]: # Histogram raspodele odbiraka po klasama
class_counts = {c: 0 for c in classes}

for _, labels in Xtrain.unbatch():
    class_counts[classes[labels.numpy()]] += 1

print('Broj odbiraka po klasama (trening skup):')
for c, count in class_counts.items():
    print(f' {c}: {count}')

plt.figure(figsize=(8, 5))
plt.bar(class_counts.keys(), class_counts.values(), color=['#4C72B0', '#DD8452'][:num_classes])
plt.xlabel('Klasa')
plt.ylabel('Broj odbiraka')
plt.title('Raspodela odbiraka po klasama (trening skup)')
for i, (c, v) in enumerate(class_counts.items()):
    plt.text(i, v + 10, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

# Provera balansiranosti
counts = list(class_counts.values())
ratio = max(counts) / min(counts) if min(counts) > 0 else float('inf')
print(f'\nOdnos najvece/najmanje klase: {ratio:.2f}')
if ratio < 1.5:
    print('Podaci su priblizno balansirani.')
```

```

else:
    print('Podaci NISU balansirani - potrebno je primeniti tehniku balansiranja.
    ↪')

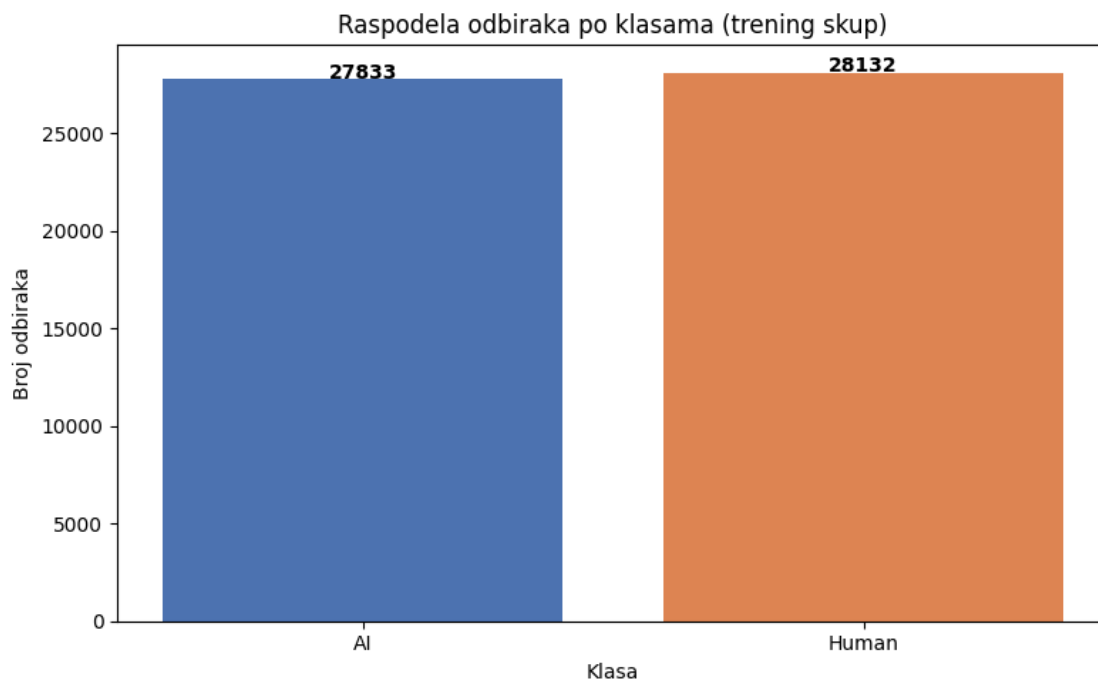
```

Broj odbiraka po klasama (trening skup):

AI: 27833

Human: 28132

2026-02-28 02:21:27.636465: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence



Odnos najveće/najmanje klase: 1.01

Podaci su približno balansirani.

```

[7]: # Racunanje class_weight u slucaju nebalansiranih klasa
from sklearn.utils.class_weight import compute_class_weight

all_labels = []
for _, labels in Xtrain.unbatch():
    all_labels.append(labels.numpy())
all_labels = np.array(all_labels)

weights = compute_class_weight('balanced', classes=np.unique(all_labels),
    ↪y=all_labels)

```

```

class_weight = dict(enumerate(weights))
print(f'Class weights (balansirani): {class_weight}')

# Rucno povecanje tezine za AI klasu (0) zbog loseg recalla
class_weight[0] *= 2.0
print(f'Class weights (korigovani): {class_weight}')

```

```

Class weights (balansirani): {0: np.float64(1.0053713218122373), 1:
np.float64(0.9946857670979667)}
Class weights (korigovani): {0: np.float64(2.0107426436244746), 1:
np.float64(0.9946857670979667)}

```

```

2026-02-28 02:21:35.371457: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

```

1.5 4. Predprocesiranje i augmentacija podataka

Primenjene transformacije: - **Skaliranje (Rescaling)**: Vrednosti piksela se normalizuju sa opsega [0, 255] na [0, 1] kako bi se ubrzala konvergencija treninga. - **Horizontalno okretanje (RandomFlip)**: Slika se nasumično okreće po horizontalnoj osi, čime se povećava raznovrsnost trening skupa. - **Nasumična rotacija (RandomRotation)**: Slika se rotira za nasumični ugao, što pomaže modelu da bude otporniji na orijentaciju objekata. - **Nasumični zum (RandomZoom)**: Slika se nasumično zumira, simulirajući različite udaljenosti kamere.

Ove augmentacije se primenjuju samo tokom treninga i pomažu u smanjenju preobucenosti (overfitting).

```

[8]: # Augmentacija podataka
data_augmentation = Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.1),
])

# Prikaz augmentiranih slika
plt.figure(figsize=(12, 6))
for images, _ in Xtrain.take(1):
    img = images[0]
    for i in range(8):
        aug_img = data_augmentation(tf.expand_dims(img, 0))
        plt.subplot(2, 4, i + 1)
        plt.imshow(aug_img[0].numpy().astype('uint8'))
        plt.axis('off')
plt.suptitle('Primeri augmentacije jedne slike', fontsize=14)
plt.tight_layout()
plt.show()

```

Primeri augmentacije jedne slike



1.6 5. Definisanje CNN modela

Projektovana je konvoluciona neuralna mreža sa 4 konvoluciona bloka.

Obrazloženje izbora: - **Kriterijumska funkcija:** `SparseCategoricalCrossentropy` — standardni izbor za klasifikaciju u više klasa gde su labele celobrojne. - **Funkcija aktivacije neurona:** `ReLU` u skrivenim slojevima (rešava problem nestajućeg gradijenta, računski efikasna), `Softmax` u izlaznom sloju (daje distribuciju verovatnoća po klasama). - **Optimizacija:** `Adam` — adaptivna metoda optimizacije koja kombinuje prednosti `AdaGrad` i `RMSProp` algoritama, dobro funkcioniše sa podrazumevanim hiperparametrima.

Zaštita od preobučavanja (overfitting):

Preobučavanje je pojava kada model previše dobro nauči trening podatke, uključujući šum i specifičnosti, pa ne uspeva da generalizuje na nove, nevidene podatke. To se manifestuje kroz veliku razliku između performansi na trening i validacionom/test skupu.

Primenjene metode zaštite: - **Dropout (0.3):** Tokom treninga, nasumično isključuje 30% neurona, čime se sprečava ko-adaptacija i forsira redundantna reprezentacija. - **Batch Normalization:** Normalizuje aktivacije unutar svakog mini-batch-a, stabilizuje trening i ima blagi regularizacioni efekat. - **Early Stopping:** Prekida trening kada se validaciona metrika prestane poboljšavati, čime se sprečava nepotrebno treniranje koje vodi ka preobučavanju.

```
[9]: def cnn_model(num_classes):  
    model = Sequential([  
        # Augmentacija i skaliranje  
        layers.RandomFlip('horizontal'),  
        layers.RandomRotation(0.15),  
        layers.RandomZoom(0.1),  
        layers.Rescaling(1./255),
```



```

# Konvolucioni blok 1
layers.Conv2D(32, 3, padding='same', activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(),

# Konvolucioni blok 2
layers.Conv2D(64, 3, padding='same', activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(),

# Konvolucioni blok 3
layers.Conv2D(128, 3, padding='same', activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(),

# Konvolucioni blok 4
layers.Conv2D(256, 3, padding='same', activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(),

# Klasifikaciona glava
layers.Dropout(0.5),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)

return model

```

```

[10]: model = cnn_model(num_classes)
      model.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
random_flip_1 (RandomFlip)	?	0 (unbuilt)

random_rotation_1 (RandomRotation)	?	0 (unbuilt)
random_zoom_1 (RandomZoom)	?	0 (unbuilt)
rescaling (Rescaling)	?	0 (unbuilt)
conv2d (Conv2D)	?	0 (unbuilt)
batch_normalization (BatchNormalization)	?	0 (unbuilt)
max_pooling2d (MaxPooling2D)	?	0
conv2d_1 (Conv2D)	?	0 (unbuilt)
batch_normalization_1 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_1 (MaxPooling2D)	?	0
conv2d_2 (Conv2D)	?	0 (unbuilt)
batch_normalization_2 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_2 (MaxPooling2D)	?	0
conv2d_3 (Conv2D)	?	0 (unbuilt)
batch_normalization_3 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_3 (MaxPooling2D)	?	0
dropout (Dropout)	?	0
flatten (Flatten)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

1.7 6. Treniranje modela

```
[11]: es = EarlyStopping(  
    monitor='val_accuracy',  
    patience=5,  
    restore_best_weights=True,  
    verbose=1  
)  
  
history = model.fit(  
    Xtrain,  
    epochs=30,  
    validation_data=Xval,  
    callbacks=[es],  
    class_weight=class_weight,  
    verbose=1  
)
```

Epoch 1/30

875/875 275s 311ms/step -
accuracy: 0.8669 - loss: 0.4560 - val_accuracy: 0.7396 - val_loss: 0.8443

Epoch 2/30

875/875 316s 361ms/step -
accuracy: 0.9060 - loss: 0.3233 - val_accuracy: 0.7646 - val_loss: 0.8174

Epoch 3/30

875/875 319s 364ms/step -
accuracy: 0.9178 - loss: 0.2818 - val_accuracy: 0.7674 - val_loss: 0.8890

Epoch 4/30

875/875 318s 363ms/step -
accuracy: 0.9260 - loss: 0.2610 - val_accuracy: 0.7551 - val_loss: 0.9000

Epoch 5/30

875/875 319s 364ms/step -
accuracy: 0.9290 - loss: 0.2470 - val_accuracy: 0.7596 - val_loss: 0.8795

Epoch 6/30

875/875 332s 379ms/step -
accuracy: 0.9335 - loss: 0.2354 - val_accuracy: 0.7770 - val_loss: 0.8299

Epoch 7/30

875/875 336s 384ms/step -
accuracy: 0.9350 - loss: 0.2268 - val_accuracy: 0.7760 - val_loss: 0.8153

Epoch 8/30

875/875 337s 385ms/step -

```

accuracy: 0.9387 - loss: 0.2128 - val_accuracy: 0.7236 - val_loss: 1.1730
Epoch 9/30
875/875          338s 386ms/step -
accuracy: 0.9401 - loss: 0.2075 - val_accuracy: 0.7707 - val_loss: 0.7421
Epoch 10/30
875/875          344s 393ms/step -
accuracy: 0.9425 - loss: 0.1991 - val_accuracy: 0.7910 - val_loss: 0.7173
Epoch 11/30
875/875          337s 386ms/step -
accuracy: 0.9442 - loss: 0.1962 - val_accuracy: 0.7881 - val_loss: 0.7248
Epoch 12/30
875/875          337s 385ms/step -
accuracy: 0.9476 - loss: 0.1864 - val_accuracy: 0.8440 - val_loss: 0.4350
Epoch 13/30
875/875          324s 370ms/step -
accuracy: 0.9471 - loss: 0.1832 - val_accuracy: 0.8113 - val_loss: 0.6476
Epoch 14/30
875/875          311s 356ms/step -
accuracy: 0.9492 - loss: 0.1772 - val_accuracy: 0.7842 - val_loss: 0.8171
Epoch 15/30
875/875          312s 356ms/step -
accuracy: 0.9500 - loss: 0.1742 - val_accuracy: 0.7772 - val_loss: 0.7780
Epoch 16/30
875/875          312s 357ms/step -
accuracy: 0.9525 - loss: 0.1670 - val_accuracy: 0.7530 - val_loss: 0.8829
Epoch 17/30
875/875          311s 355ms/step -
accuracy: 0.9520 - loss: 0.1664 - val_accuracy: 0.7804 - val_loss: 0.7946
Epoch 17: early stopping
Restoring model weights from the end of the best epoch: 12.

```

```
[12]: model.save('cnn_model.keras')
      print('Model sacuvan.')
```

Model sacuvan.

```
[13]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
random_flip_1 (RandomFlip)	(None, 128, 128, 3)	0
random_rotation_1 (RandomRotation)	(None, 128, 128, 3)	0

random_zoom_1 (RandomZoom)	(None , 128, 128, 3)	0
rescaling (Rescaling)	(None , 128, 128, 3)	0
conv2d (Conv2D)	(None , 128, 128, 32)	896
batch_normalization (BatchNormalization)	(None , 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(None , 64, 64, 32)	0
conv2d_1 (Conv2D)	(None , 64, 64, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None , 64, 64, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None , 32, 32, 64)	0
conv2d_2 (Conv2D)	(None , 32, 32, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None , 32, 32, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None , 16, 16, 128)	0
conv2d_3 (Conv2D)	(None , 16, 16, 256)	295,168
batch_normalization_3 (BatchNormalization)	(None , 16, 16, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None , 8, 8, 256)	0
dropout (Dropout)	(None , 8, 8, 256)	0
flatten (Flatten)	(None , 16384)	0
dense (Dense)	(None , 128)	2,097,280
dropout_1 (Dropout)	(None , 128)	0
dense_1 (Dense)	(None , 2)	258

Total params: 7,461,704 (28.46 MB)

Trainable params: 2,486,914 (9.49 MB)

Non-trainable params: 960 (3.75 KB)

Optimizer params: 4,973,830 (18.97 MB)

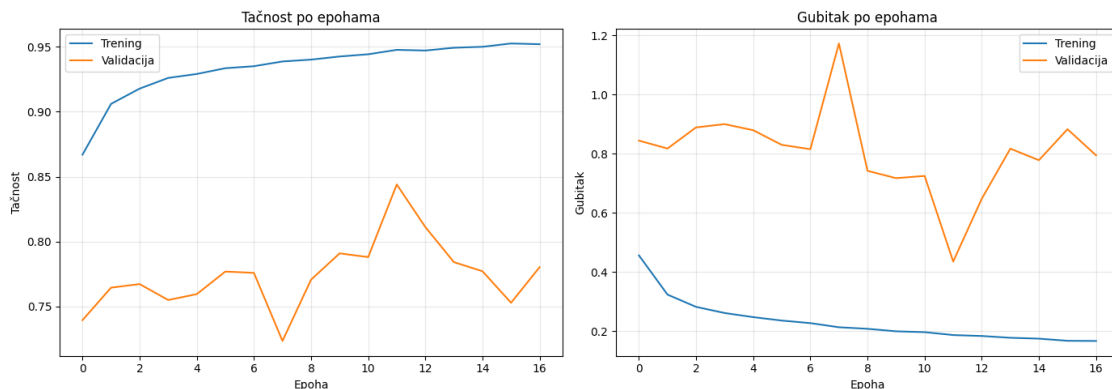
1.8 7. Grafik promena performansi tokom epoha treniranja

```
[14]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Tačnost
ax1.plot(history.history['accuracy'], label='Trening')
ax1.plot(history.history['val_accuracy'], label='Validacija')
ax1.set_title('Tačnost po epohama')
ax1.set_xlabel('Epoha')
ax1.set_ylabel('Tačnost')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Gubitak
ax2.plot(history.history['loss'], label='Trening')
ax2.plot(history.history['val_loss'], label='Validacija')
ax2.set_title('Gubitak po epohama')
ax2.set_xlabel('Epoha')
ax2.set_ylabel('Gubitak')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



1.9 8. Evaluacija modela

```
[15]: # Predikcije na test skupu
y_true_test = np.array([])
y_pred_test = np.array([])
for img, lab in Xtest:
    y_true_test = np.concatenate([y_true_test, lab.numpy()])
    y_pred_test = np.concatenate([y_pred_test, np.argmax(model.predict(img,
↪ verbose=0), axis=1)])

print(f'Tačnost modela na test skupu: {100 * accuracy_score(y_true_test,
↪ y_pred_test):.2f}%')
print()
print('Detaljan izveštaj:')
print(classification_report(y_true_test, y_pred_test, target_names=classes))
```

Tačnost modela na test skupu: 84.72%

Detaljan izveštaj:

	precision	recall	f1-score	support
AI	0.98	0.71	0.82	8072
Human	0.77	0.99	0.86	7918
accuracy			0.85	15990
macro avg	0.88	0.85	0.84	15990
weighted avg	0.88	0.85	0.84	15990

2026-02-28 03:53:24.240896: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

```
[16]: # Predikcije na trening skupu
y_true_train = np.array([])
y_pred_train = np.array([])
for img, lab in Xtrain:
    y_true_train = np.concatenate([y_true_train, lab.numpy()])
    y_pred_train = np.concatenate([y_pred_train, np.argmax(model.predict(img,
↪ verbose=0), axis=1)])

print(f'Tačnost modela na trening skupu: {100 * accuracy_score(y_true_train,
↪ y_pred_train):.2f}%')
```

Tačnost modela na trening skupu: 85.36%

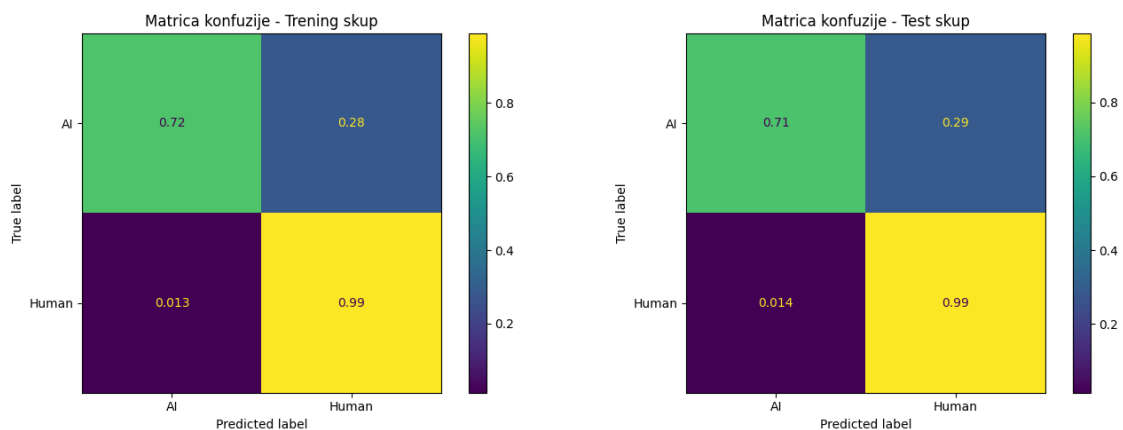
1.10 9. Matrica konfuzije

```
[17]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Matrica konfuzije - trening skup
cm_train = confusion_matrix(y_true_train, y_pred_train, normalize='true')
ConfusionMatrixDisplay(confusion_matrix=cm_train, display_labels=classes).
    plot(ax=ax1)
ax1.set_title('Matrica konfuzije - Trening skup')

# Matrica konfuzije - test skup
cm_test = confusion_matrix(y_true_test, y_pred_test, normalize='true')
ConfusionMatrixDisplay(confusion_matrix=cm_test, display_labels=classes).
    plot(ax=ax2)
ax2.set_title('Matrica konfuzije - Test skup')

plt.tight_layout()
plt.show()
```



1.11 10. Primeri dobro i loše klasifikovanih slika

```
[18]: # Prikupljanje slika, pravih labela i predikcija sa test skupa
all_images = []
all_true = []
all_pred = []

for img_batch, lab_batch in Xtest:
    preds = np.argmax(model.predict(img_batch, verbose=0), axis=1)
    for i in range(len(lab_batch)):
        all_images.append(img_batch[i].numpy().astype('uint8'))
        all_true.append(lab_batch[i].numpy())
        all_pred.append(preds[i])
```



```

all_true = np.array(all_true)
all_pred = np.array(all_pred)

correct_idx = np.where(all_true == all_pred)[0]
incorrect_idx = np.where(all_true != all_pred)[0]

print(f'Ukupno tačno klasifikovanih: {len(correct_idx)}')
print(f'Ukupno pogrešno klasifikovanih: {len(incorrect_idx)}')

```

Ukupno tačno klasifikovanih: 13547

Ukupno pogrešno klasifikovanih: 2443

```

[19]: # Dobro klasifikovane slike
n_show = min(8, len(correct_idx))
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
sample = np.random.choice(correct_idx, n_show, replace=False)
for i, idx in enumerate(sample):
    ax = axes[i // 4][i % 4]
    ax.imshow(all_images[idx])
    ax.set_title(f'Stvarno: {classes[all_true[idx]]}\nPredikcija: ↵
    ↵{classes[all_pred[idx]]}', fontsize=10)
    ax.axis('off')
plt.suptitle('Primeri DOBRO klasifikovanih slika', fontsize=14)
plt.tight_layout()
plt.show()

```



```
[ ]: 875/875 ██████████ 337s 385ms/step - accuracy: 0.9387 - loss: 0.2128 -
    ↳ val_accuracy: 0.7236 - val_loss: 1.1730
if n_show > 0:
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    sample = np.random.choice(incorrect_idx, n_show, replace=False)
    for i, idx in enumerate(sample):
        ax = axes[i // 4][i % 4]
        ax.imshow(all_images[idx])
        ax.set_title(f'Stvarno: {classes[all_true[idx]]}\nPredikcija: ↳
    ↳ {classes[all_pred[idx]]}', fontsize=10, color='red')
        ax.axis('off')
    # Sakrij prazne subplotove+*****
    for i in range(n_show, 8):
        axes[i // 4][i % 4].axis('off')
    plt.suptitle('Primeri POGREŠNO klasifikovanih slika', fontsize=14)
    plt.tight_layout()
    plt.show()
else:
    print('Nema pogrešno klasifikovanih slika!')
```

